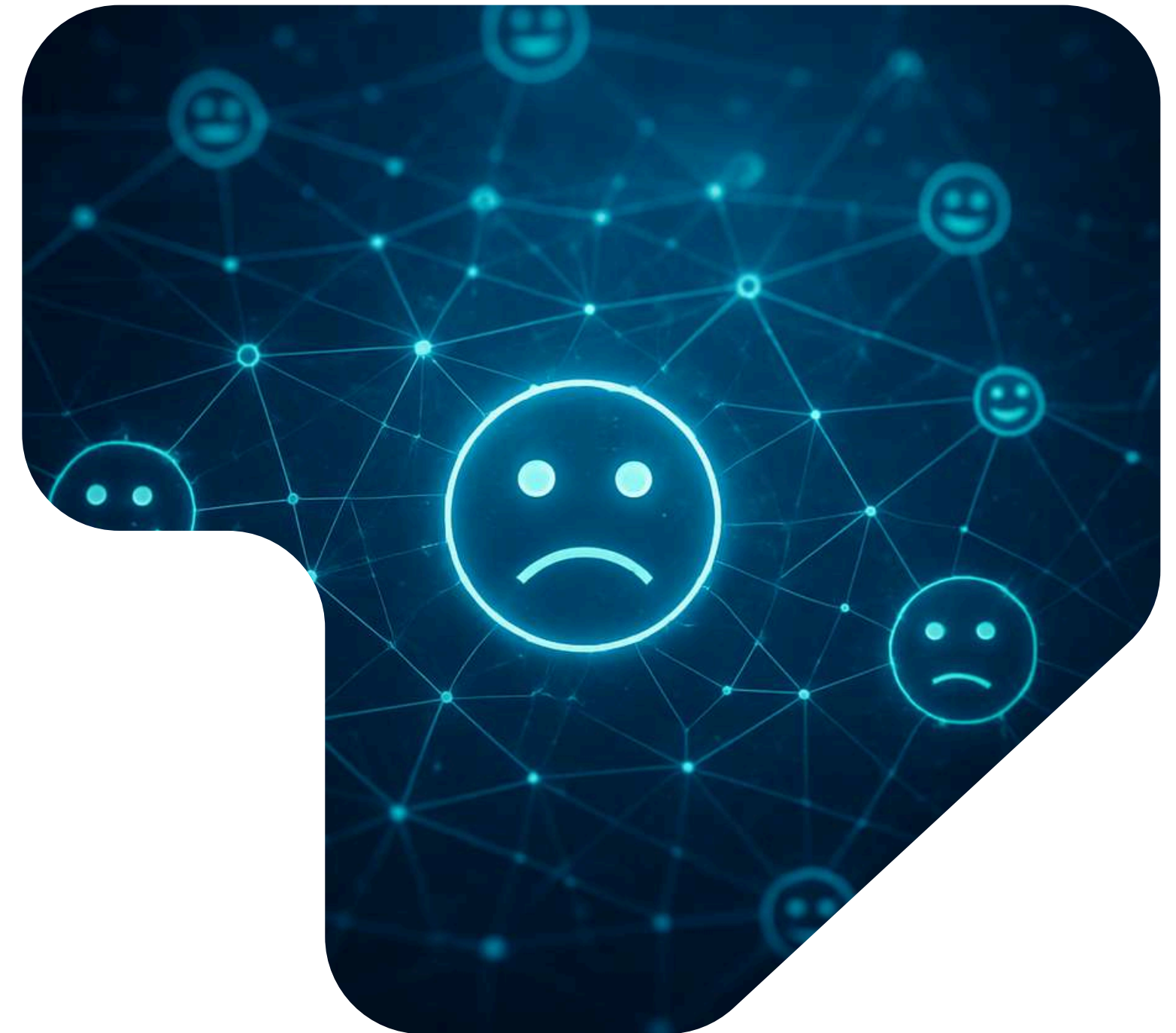


Análisis de sentimientos

Integrantes:

- Miquel Leopoldo Muñiz
- Marco Adrian Zambrano



¿Qué es el análisis de sentimientos?



El Análisis de Sentimientos es una técnica del Procesamiento de Lenguaje Natural (NLP) que permite identificar, extraer y clasificar emociones u opiniones expresadas en textos.

En palabras simples:

Una computadora lee texto y decide si el sentimiento es positivo, negativo o neutral (y a veces emociones más específicas como alegría, enojo, tristeza).



Ejemplos:

- “Este producto es increíble” → positivo
- “El servicio fue pésimo” → negativo
- “El paquete llegó hoy” → neutral

Contexto

El análisis de sentimientos
surge por las siguientes
razones



Explosión de texto en internet

De repente la gente empezó a escribir todo en redes sociales, foros, reseñas, blogs, WhatsApp, etc. Mucho texto, cero estructura. Leer eso a mano: misión imposible.

Tomar decisiones basadas en emociones reales

Las decisiones ya no podían basarse solo en números. El sentimiento revela cosas que las métricas no muestran: frustración, satisfacción, enojo, hype.

Las empresas necesitaban saber qué siente la gente

No bastaba con saber qué decían, sino cómo lo decían.

- ¿Están felices con el producto?
- ¿Están tirando hate?
- ¿Se viene una crisis de reputación?



Importancia de la visualización de datos

La visualización de datos cumple un rol fundamental en el análisis de sentimientos dentro de la minería de datos, ya que permite interpretar y comunicar de forma efectiva los resultados obtenidos a partir de modelos de procesamiento de lenguaje natural (NLP).

Mediante técnicas de visualización, los valores numéricos generados por los modelos (scores de polaridad, probabilidades de clase, distribuciones de sentimiento) se transforman en representaciones gráficas que facilitan la identificación de patrones, tendencias temporales y comportamientos anómalos en grandes volúmenes de datos textuales.

Además, la visualización permite evaluar el impacto de eventos específicos, comparar resultados entre periodos o categorías y apoyar la toma de decisiones estratégicas por parte de usuarios no técnicos. De esta forma, se reduce la complejidad del modelo subyacente y se mejora la comprensión de los resultados analíticos.



Visualización de datos

Ejemplos:



Gráficos de barras

Se utilizan gráficos de barras para representar la frecuencia absoluta o relativa de cada clase de sentimiento generada por el modelo.

Pipeline de datos

1. Entrada: textos (reviews, tweets, comentarios)
2. Procesamiento NLP + modelo entrenado
3. Output: etiqueta de sentimiento por texto
4. Agregación: COUNT(sentimiento)
5. Visualización: barras



Líneas de tiempo

Se utiliza una serie temporal para analizar cómo varía el sentimiento agregado en función del tiempo. Aquí no importa solo el sentimiento, sino el timestamp asociado al texto.

Pipeline de datos

1. Texto + fecha/hora
2. Predicción de sentimiento
3. Conversión a score numérico
4. Agrupación por: día, semana, mes
5. Cálculo: AVG(sentiment_score)

Ejemplos:



Mapas de calor (Heatmaps)

Un mapa de calor representa la intensidad del sentimiento promedio por región geográfica.

Requiere metadatos espaciales:

- País
- Ciudad
- Coordenadas (lat, long)

Pipeline de datos

1. Texto
2. Predicción de sentimiento
3. Asociación geográfica
4. Agregación por región: `AVG(sentiment_score) GROUP BY region`
5. Visualización en mapa

Diagramas de dispersión (Scatter Plot)

Se usan para visualizar la relación entre:

- Score de sentimiento
- Probabilidad / confianza del modelo

Cada punto representa un texto.

Ejes típicos

- Eje X → `sentiment_score` (-1 a 1)
- Eje Y → `confidence` (0 a 1)

Ejemplo técnico

Un cluster cerca de 0 indica textos neutrales o confusos.

Pipeline de datos

1. Adquisición de datos
 - Tweets
 - Reviews
 - Comentarios
 - Chats
- Preprocesamiento NLP
- Vectorización

Se transforma el texto en vectores:

 - TF-IDF
 - Embeddings (Word2Vec, BERT, etc.)
- Predicción del modelo
- Construcción del dataset para visualización
- Visualización (Scatter Plot)

PREPROCESAMIENTO DE TEXTO

Antes de analizar sentimientos, el texto es un desastre:
mayúsculas, emojis, errores, etc.
Por eso se limpia.

Limpieza de texto

Consiste en eliminar ruido:

- Signos innecesarios
- URLs
- Emojis (dependiendo del caso)
- Números irrelevantes
- Convertir a minúsculas

Ejemplo:

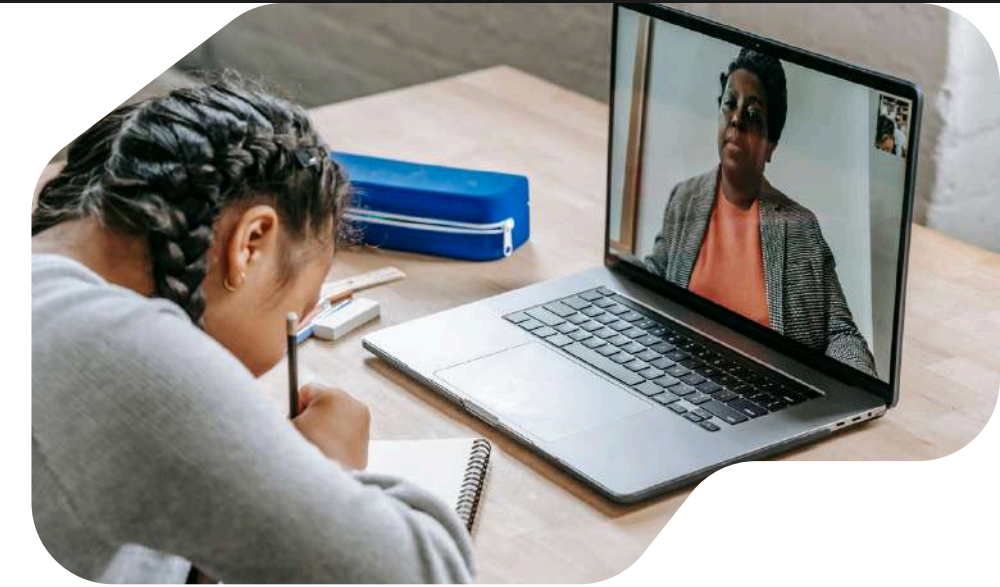
Texto original:

"Excelente servicio!!! 🥰🥰🥰 www.ejemplo.com"

Texto limpio:

"excelente servicio"

Conceptos, Técnicas y Herramientas a tener en cuenta



Tokenización

Es dividir el texto en partes pequeñas llamadas tokens (normalmente palabras).

Ejemplo:

"me encanta este producto"
→ ["me", "encanta", "este",
"producto"]

La máquina no entiende frases,
entiende tokens.

Stopwords

Son palabras muy comunes que no aportan significado emocional:

- el, la, de, que, y, a, en...

Ejemplo:

["me", "encanta", "este", "producto"]
→ ["encanta", "producto"]

Esto mejora rendimiento y
precisión.

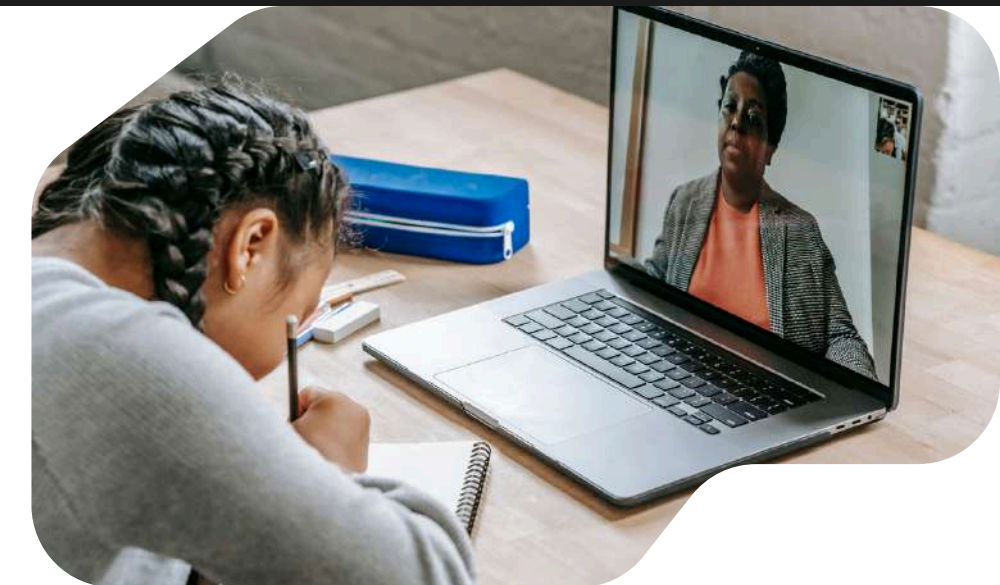
Entrenaremos a la IA con estos 5 ejemplos etiquetados (1=Positivo, 0=Negativo).

```
import pandas as pd
```

```
# Creamos un dataset pequeño de ejemplo
```

```
data = [  
    ("¡Me encanta este producto, es genial!", 1), # Positivo  
    ("El servicio fue pésimo y lento.", 0),      # Negativo  
    ("Llegó a tiempo y funciona bien.", 1),      # Positivo  
    ("No me gustó, mala calidad.", 0),           # Negativo  
    ("Estoy muy feliz con la compra.", 1)         # Positivo  
]
```

```
df = pd.DataFrame(data, columns=['texto', 'sentimiento'])  
print(df)
```



```
Generate + Code + Markdown  
import pandas as pd  
  
# Creamos un dataset pequeño de ejemplo  
data = [  
    ("¡Me encanta este producto, es genial!", 1), # Positivo  
    ("El servicio fue pésimo y lento.", 0),      # Negativo  
    ("Llegó a tiempo y funciona bien.", 1),      # Positivo  
    ("No me gustó, mala calidad.", 0),           # Negativo  
    ("Estoy muy feliz con la compra.", 1)         # Positivo  
]  
  
df = pd.DataFrame(data, columns=['texto', 'sentimiento'])  
print(df)  
✓ 14.9s
```

	texto	sentimiento
0	¡Me encanta este producto, es genial!	1
1	El servicio fue pésimo y lento.	0
2	Llegó a tiempo y funciona bien.	1
3	No me gustó, mala calidad.	0
4	Estoy muy feliz con la compra.	1

Convertimos el texto sucio en una lista de palabras útiles.

```
import re
```

```
texto_original = "¡Me encanta este producto, es genial!"
```

```
# 1. Limpieza (Quitar signos y minúsculas)
```

```
texto_limpio = re.sub(r'^\w\s', '', texto_original).lower()
```

```
# 2. Tokenización (Dividir en palabras)
```

```
tokens = texto_limpio.split()
```

```
# 3. Stopwords (Quitar palabras vacías manualmente para el ejemplo)
```

```
stopwords = {'me', 'este', 'es', 'el', 'la', 'y'}
```

```
tokens_filtrados = [w for w in tokens if w not in stopwords]
```

```
print(f"Original: {texto_original}")
```

```
print(f"Procesado: {tokens_filtrados}")
```



```
import re

texto_original = "¡Me encanta este producto, es genial!"

# 1. Limpieza (Quitar signos y minúsculas)
texto_limpio = re.sub(r'^\w\s', '', texto_original).lower()

# 2. Tokenización (Dividir en palabras)
tokens = texto_limpio.split()

# 3. Stopwords (Quitar palabras vacías manualmente para el ejemplo)
stopwords = {'me', 'este', 'es', 'el', 'la', 'y'}
tokens_filtrados = [w for w in tokens if w not in stopwords]

print(f"Original: {texto_original}")
print(f"Procesado: {tokens_filtrados}")

✓ 0.0s
```

```
Original: ¡Me encanta este producto, es genial!
Procesado: ['encanta', 'producto', 'genial']
```

Lematización / Stemming

Sirve para reducir palabras a su forma base.

- Stemming → corta la palabra (más rápido, menos preciso)
- Lematización → busca la forma correcta del diccionario

Ejemplo:

"corriendo", "corrí", "correr"

→ "correr"

Así el modelo no aprende lo mismo 20 veces.

VECTORIZACIÓN

Las máquinas no entienden texto, solo números.
Aquí ocurre la magia.

Bolsa de Palabras (Bag of Words)

Cuenta cuántas veces aparece cada palabra.

Ejemplo:

T1: "me gusta nlp"

T2: "nlp es genial"

Vocabulario: ['me', 'gusta', 'nlp', 'es', 'genial']

Vectores:

T1 → [1, 1, 1, 0, 0]

T2 → [0, 0, 1, 1, 1]

Ventaja:

- Fácil de entender

Limitación:

- No entiende contexto ni importancia



Opción A: Bolsa de Palabras (Bag of Words) Cuenta cuántas veces aparece cada palabra.

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
vectorizer = CountVectorizer()
```

```
# Transformamos nuestros textos a números
```

```
matriz_bow = vectorizer.fit_transform(df['texto'])
```

```
# Mostramos el vocabulario aprendido y el vector de la primera frase
```

```
print("Vocabulario:", vectorizer.get_feature_names_out())
```

```
print("Vector Frase 1:", matriz_bow[0].toarray())
```



```
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer()
# Transformamos nuestros textos a números
matriz_bow = vectorizer.fit_transform(df['texto'])

# Mostramos el vocabulario aprendido y el vector de la primera frase
print("Vocabulario:", vectorizer.get_feature_names_out())
print("Vector Frase 1:", matriz_bow[0].toarray())
```

[7] ✓ 27.6s

```
... Vocabulario: ['bien' 'calidad' 'compra' 'con' 'el' 'encanta' 'es' 'este' 'estoy'
'feliz' 'fue' 'funciona' 'genial' 'gustó' 'la' 'lento' 'llegó' 'mala'
'me' 'muy' 'no' 'producto' 'pésimo' 'servicio' 'tiempo']
Vector Frase 1: [[0 0 0 0 1 1 1 0 0 0 0 1 0 0 0 0 1 0 0 1 0 0 0]]
```


TF-IDF

Mejora Bag of Words.

- TF (Term Frequency) → frecuencia en el texto
- IDF (Inverse Document Frequency) → penaliza palabras muy comunes

Resultado:

- Palabras importantes tienen mayor peso
- Palabras genéricas pesan menos

Es MUY usado en análisis de sentimientos clásico.



Word Embeddings (Word2Vec, GloVe)

Convierte palabras en vectores que capturan significado y contexto.

Ejemplo:

- “bueno” y “excelente” → vectores cercanos
- “bueno” y “horrible” → vectores lejanos

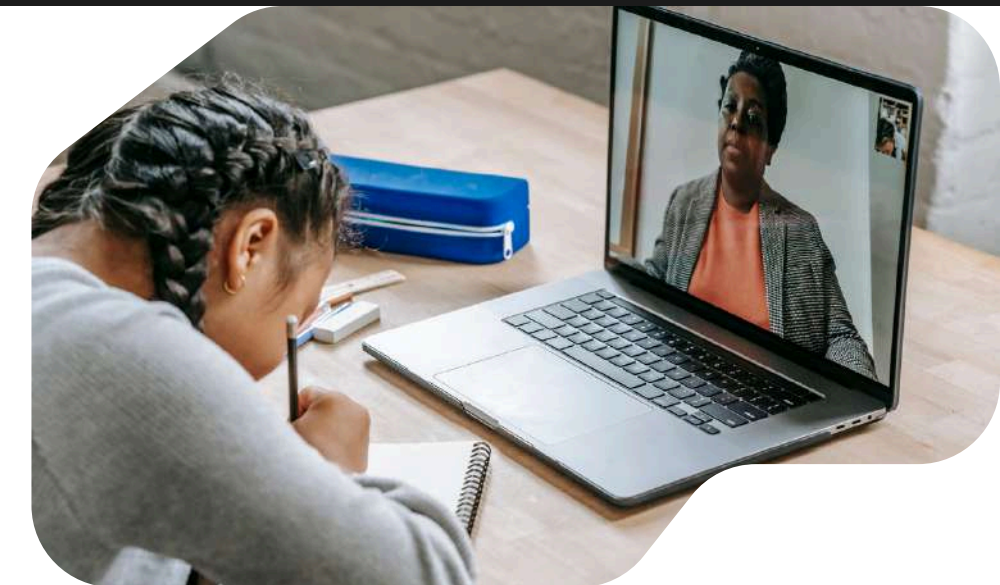
Ventajas:

- Entiende relaciones semánticas
- Mucho más preciso

Limitación:

- Más complejo y pesado





Opción B: Word Embeddings (Avanzado) Esto es para la diapositiva de "Word2Vec". Como requiere modelos pesados, muestras el código de cómo se haría.

```
import gensim.downloader as api
```

```
# Descargamos un modelo pre-entrenado (ej. Twitter)
```

```
# Esto convierte palabras en coordenadas matemáticas
```

```
modelo_w2v = api.load("glove-twitter-25")
```

```
# Verificamos si la máquina entiende la relación
```

```
vector_rey = modelo_w2v['king']
```

```
vector_reina = modelo_w2v['queen']
```

```
# La magia: Rey - Hombre + Mujer = ¿Reina?
```

```
resultado =
```

```
modelo_w2v.most_similar(positive=['woman', 'king'], negative=['man'])
```

```
print(resultado[0]) # Debería decir 'queen'
```

```
import gensim.downloader as api

# Descargamos un modelo pre-entrenado (ej. Twitter)
# Esto convierte palabras en coordenadas matemáticas
modelo_w2v = api.load("glove-twitter-25")

# Verificamos si la máquina entiende la relación
vector_rey = modelo_w2v['king']
vector_reina = modelo_w2v['queen']

# La magia: Rey - Hombre + Mujer = ¿Reina?
❖ resultado = modelo_w2v.most_similar(positive=['woman', 'king'], negative=['man'])
print(resultado) # Debería decir 'queen'

✓ 50.0s Python

[('meets', 0.8841923475265503), ('prince', 0.832163393497467), ('queen', 0.8257461190223694), (''s', 0.81740
```

ENTRENAMIENTO DEL MODELO (NLP)

¿Qué aprende el modelo?

El modelo recibe:

- $X \rightarrow$ texto vectorizado
- $Y \rightarrow$ etiqueta de sentimiento (positivo, negativo, neutral)

Ejemplo:

"me encanta este producto" $\rightarrow$ positivo

"no recomiendo este servicio" $\rightarrow$ negativo

El modelo aprende patrones entre:

- Palabras
- Frecuencias
- Contextos
- y su sentimiento asociado.

Modelos comunes

- Naive Bayes
- Logistic Regression
- SVM
- Redes neuronales
- Transformers (BERT, RoBERTa)

Procesamiento de Lenguaje Natural (NLP)

Entrenar un modelo NLP consiste en aprender una función matemática que mapea:

$$f: \mathbb{R}^n \rightarrow y$$

Donde:

$\mathbb{R}^n$

- $\rightarrow$ espacio vectorial del texto (TF-IDF, embeddings, etc.)
- $y \rightarrow$ conjunto de etiquetas de sentimiento (positivo, negativo, neutral)

El modelo no aprende palabras, aprende patrones numéricos asociados a sentimientos.



Datos de entrenamiento

Después de vectorizar, el dataset queda así:

Matriz de características (X)

Cada fila es un texto vectorizado.

$X = \begin{bmatrix} [0.21, 0.00, 0.83, 0.12, \dots], \\ [0.05, 0.77, 0.00, 0.44, \dots], \\ [0.91, 0.02, 0.00, 0.01, \dots] \end{bmatrix}$

Vector de etiquetas (y)

$y = [1, 0, 1]$

Ejemplo:

- 1 $\rightarrow$ positivo
- 0 $\rightarrow$ negativo

Modelo de Ejemplo: Regresión Logística

La regresión logística aprende pesos (w) para cada dimensión del vector.

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Luego aplica la función sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Esto produce una probabilidad:

$$P(y=1/x)$$

Interpretación Resultante

- Palabras como “excelente” → peso positivo alto
- Palabras como “pésimo” → peso negativo
- Palabras neutras → peso cercano a 0

El modelo aprende qué palabras o combinaciones empujan la decisión hacia cada sentimiento

Proceso de aprendizaje

Durante el entrenamiento:

- El modelo hace una predicción
- Compara con la etiqueta real
- Calcula el error (función de pérdida, ej. log loss)
- Ajusta los pesos con gradient descent
- Repite hasta minimizar el error

Precisión (Accuracy)

La precisión (accuracy) mide la proporción de predicciones correctas respecto al total de predicciones realizadas.

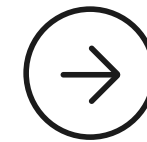
Recall (Sensibilidad o Exhaustividad)

El recall mide la capacidad del modelo para detectar correctamente los casos positivos reales de una clase específica.

En análisis de sentimientos, normalmente se calcula por clase, por ejemplo para la clase negativo.

Precisión por clase (Precision)

La precision mide qué proporción de las predicciones positivas realizadas por el modelo son realmente correctas.



EVALUACIÓN DEL MODELO

La evaluación del modelo consiste en medir qué tan bien un modelo de Machine Learning predice los sentimientos, comparando sus predicciones con las etiquetas reales del conjunto de prueba.

F1-Score

El F1-Score es la media armónica entre la precisión (precision) y el recall, y permite evaluar el equilibrio entre ambas métricas.

formula:

$$F1 = 2 * (precision * recall) / (precision + recall)$$

Enseñamos al algoritmo.

```
from sklearn.linear_model import
LogisticRegression
from sklearn.feature_extraction.text import
TfidfVectorizer

# 1. Vectorizamos con TF-IDF (Más avanzado que
Bag of Words)
tfidf = TfidfVectorizer(stop_words=['el', 'la', 'y', 'de'])
X = tfidf.fit_transform(df['texto'])
y = df['sentimiento']

# 2. Entrenamos el modelo (Regresión Logística)
modelo = LogisticRegression()
modelo.fit(X, y)

print("¡Modelo entrenado exitosamente!")
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.feature_extraction.text import TfidfVectorizer
```

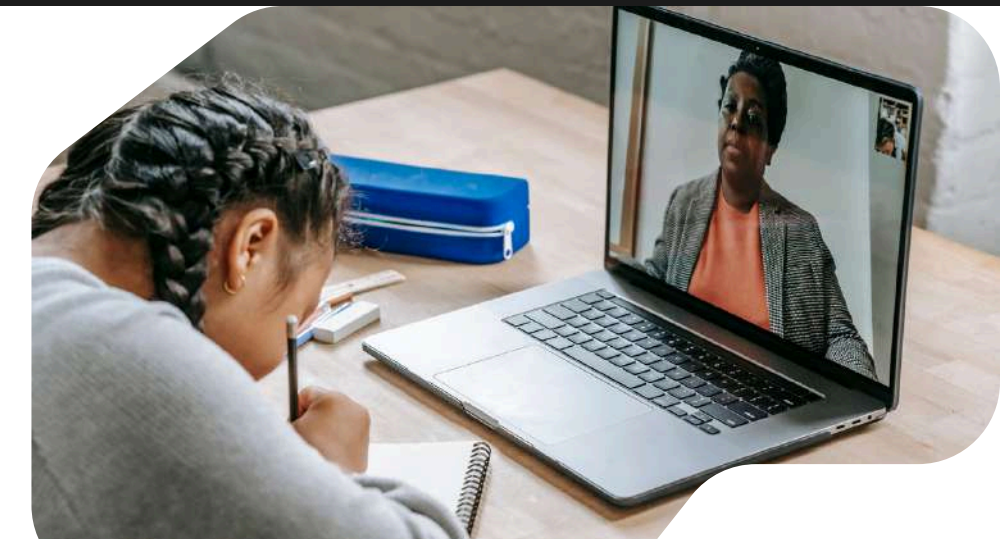
```
# 1. Vectorizamos con TF-IDF (Más avanzado que Bag of Words)
tfidf = TfidfVectorizer(stop_words=['el', 'la', 'y', 'de'])
X = tfidf.fit_transform(df['texto'])
y = df['sentimiento']
```

```
# 2. Entrenamos el modelo (Regresión Logística)
modelo = LogisticRegression()
modelo.fit(X, y)
```

```
print("¡Modelo entrenado exitosamente!")
```

✓ 2.9s

¡Modelo entrenado exitosamente!



Predicción

La predicción es la aplicación del modelo entrenado sobre datos no vistos, utilizando exactamente el mismo pipeline que en el entrenamiento.



Ejemplo de Pipeline completo de predicción:

- Entrada de texto nuevo:
Resultado: "El servicio fue lento y poco amable"
- Preprocesamiento:
 - Limpieza
 - Tokenización
 - Stopwords
 - Lematización**Resultado:** ["servicio", "lento", "poco", "amable"]
- Vectorización (MISMO vectorizador entrenado)
Ejemplo TF-IDF:
[0.00, 0.61, 0.73, 0.12, ...]
No se vuelve a entrenar el vectorizador, solo se transforma.
- Inferencia del modelo:
El modelo calcula:
 - $P(\text{negativo} \mid x) = 0.87$
 - $P(\text{positivo} \mid x) = 0.13$
- Decisión final
Según un umbral (típicamente 0.5):
 - Sentimiento → Negativo
 - Confianza → 87%

Escenario: Llega un cliente nuevo y escribe una reseña.



```
# Nuevo comentario que llega al sistema
nuevo_comentario = ["El servicio fue pésimo y lento"]

# 1. Convertimos el texto a números (usando el
MISMO transformador)
nuevo_vector = tfidf.transform(nuevo_comentario)

# 2. El modelo predice
prediccion = modelo.predict(nuevo_vector)
probabilidad =
modelo.predict_proba(nuevo_vector)

if prediccion[0] == 1:
    print("Resultado: 😊 Positivo")
else:
    print("Resultado: 😡 Negativo")

print(f"Confianza: {probabilidad.max():.2f}")
```

```
# Nuevo comentario que llega al sistema
nuevo_comentario = ["El servicio fue pésimo y lento"]

# 1. Convertimos el texto a números (usando el MISMO transformador)
nuevo_vector = tfidf.transform(nuevo_comentario)

# 2. El modelo predice
prediccion = modelo.predict(nuevo_vector)
probabilidad = modelo.predict_proba(nuevo_vector)

if prediccion[0] == 1:
    print("Resultado: 😊 Positivo")
else:
    print("Resultado: 😡 Negativo")

print(f"Confianza: {probabilidad.max():.2f}")

[30] ✓ 0.0s

... Resultado: 😡 Negativo
Confianza: 0.52
```


Ejemplos

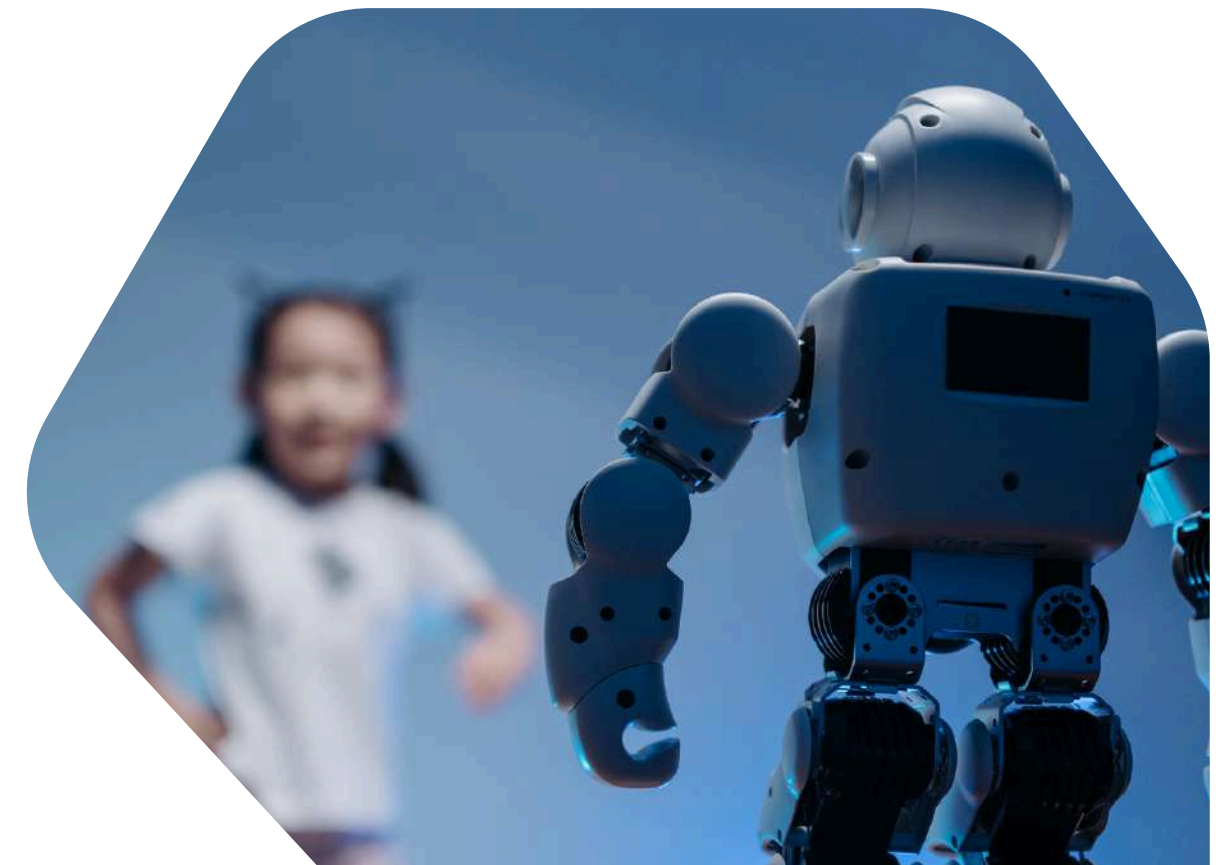
Industria

- **Análisis de reseñas y opiniones de usuarios:** permite Detectar fallas recurrentes en productos, Comparar versiones o marcas, Priorizar mejoras basadas en feedback negativo.
- **Atención al cliente automatizada:** Reducción de tiempos de respuesta, Mejor experiencia del usuario, Optimización de recursos humanos.
- **Marketing digital y reputación de marca:** Se generan series temporales de sentimiento, Se visualizan tendencias y picos anómalos, Se comparan métricas antes y después de campañas.

Investigación

- **Análisis de opinión pública:** En ciencias sociales y políticas, el análisis de sentimientos se utiliza para estudiar la percepción de la población respecto a: Políticas públicas, Figuras políticas, Eventos sociales
- **Estudios psicológicos y emocionales:** En psicología computacional, el análisis de sentimientos permite: Estudiar patrones emocionales en textos escritos, Analizar estados de ánimo en diarios digitales o redes sociales, Detectar indicadores de estrés, ansiedad o depresión.
- **Análisis de discursos y lingüística computacional:** Se utiliza para examinar: Discursos políticos, Artículos periodísticos, Comunicados institucionales

Aplicaciones en la industria y la investigación



Ventajas, Limitaciones y Buenas Prácticas



Ventajas

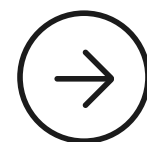
- Automatiza análisis masivo
- Ahorra tiempo y dinero
- Escalable
- Objetivo

Limitaciones

- Ironía y sarcasmo
- Ambigüedad
- Dependencia de datos de entrenamiento
- Idiomas y jerga local

Buenas prácticas

- Buen preprocesamiento
- Dataset balanceado
- Evaluar con varias métricas
- Ajustar el modelo al dominio



Conclusiones

- El análisis de sentimientos permite transformar texto en información valiosa.
- Es clave para la visualización de datos emocionales.
- Combina NLP, Machine Learning y análisis estadístico.
- Su correcta aplicación depende de buen preprocesamiento y evaluación.
- Es una herramienta poderosa, pero no infalible.

Análisis de sentimientos